Descriptive Statistics — R Practical

Companion to the Part 2 slides · 307312 Applied Statistics — Masters Level

Practical PDF · Student RStudio project · Part 2 slides

Before we begin

- Use this notebook after explaining the descriptive-statistics slides, or return to the matching section after each topic.
- Work through the code cells from top to bottom in RStudio. Run the import cell first; later cells reuse its `students` data frame.
- These examples use base R. No additional R package is needed to run the analysis cells; rendering the notebook requires Quarto, knitr and rmarkdown.
- Each section identifies the matching source slide, introduces the calculation and explains how to read its output. The worked code and its original comments are retained in full.
- Before running a cell, predict what it will display. Afterwards, connect the result to the corresponding table, chart or definition in the slides.

The data and our purpose

- **Source:** the original 307102 Descriptive Statistics teaching material used for Part 2; the original PDF, PowerPoint and workbook remain unchanged reference files.
- **Data:** student CSV and data guide, containing 8,239 records and six variables.
- **Unit:** one record in the supplied student example, rather than a product line, order or customer.
- **Categorical variables:** `gender`, `country` and `major`, with the original labels retained.
- **Numerical variables:** `age` in years, `height` in centimetres and `weight` in kilograms.
- **Limit:** original data-collection provenance has not been established. Use this as a supplied teaching dataset; do not describe it as verified records of actual university students or a random sample of a wider population.
- **Goal:** reproduce the descriptive R examples accompanying the slides, covering tables, charts, centre, spread, association, shape, quartiles and potential outliers. The Excel examples remain in the slide lesson.

1. Import and inspect the student data

- **Slides:** source slide 16.
- **What we are doing:** Load the same student dataset used in the descriptive slides. Each row is one record; the six variables are gender, age, height, weight, country and major. The first six rows let us check the column names and see how the values are recorded.
- **How to read the result:** Run this cell first. It creates `students`, which the later examples use. `data_dir` finds the CSV from either the course root or this notebook folder. Height is in centimetres, weight in kilograms and age in years. No names or student IDs are included.

```
# read.csv() imports the supplied CSV; stringsAsFactors=FALSE keeps category labels as text.
# head() previews six rows; height is in cm, weight in kg, and age in years.
# Find the shared data folder from either the course root or this notebook folder.
data_dir <- if (dir.exists("data")) "data" else file.path("..", "data")
students <- read.csv(file.path(data_dir, "descriptive-statistics/students.csv"), stringsAsFactors =
        FALSE)
head(students)
```

```
  gender age height weight country                 major
1 Female  19    160   64.8  Turkey      Political Science
2 Female  19    172   73.0   Other        Social Sciences
3 Female  22    168   70.6      UK        Social Sciences
4   Male  19    183   79.7   Other Environmental Sciences
5 Female  21    175   71.4   Italy Environmental Sciences
6   Male  19    189   85.8   Italy      Political Science
```

2. Frequency tables and percentages

- **Slides:** source slide 17.
- **What we are doing:** Describe the country categories with counts and percentages. A frequency is the number of records in a category; its percentage uses all records counted in the country table as the denominator.
- **How to read the result:** Read the two outputs together: the first gives counts and the second gives percentages. Countries are nominal categories, so a cumulative total would depend on an arbitrary ordering rather than a meaningful progression.

```r
# table() counts students in each country category; prop.table() divides by all counted students.
# Multiplying by 100 gives percentages; cumsum() is meaningful only when categories have an order.
country_counts <- table(students$country)
country_counts
```

```
Italy  Other  Russa Turkey     UK
 2797   2688    585    330   1839
```

```r
100 * prop.table(country_counts)
```

```
    Italy     Other     Russa    Turkey        UK
33.948295 32.625319  7.100376  4.005340 22.320670
```

3. Group numerical values into intervals

- **Slides:** source slide 18.
- **What we are doing:** Turn individual ages into age bands so that a frequency table can summarize a numerical variable. The selected boundaries are an added R illustration rather than a claim that these are the only suitable bands.
- **How to read the result:** The bands are [0,20), [20,25), [25,30) and [30,Inf). With `right = FALSE`, age 20 enters the second band and age 25 enters the third. Empty intervals remain visible; compare this grouped description with the original individual ages.

```r
# cut() converts ages in years into intervals; right=FALSE includes the lower boundary, not the upper.
# table() counts each interval; these explicitly chosen bands are an added R illustration.
age_groups <- cut(students$age, breaks = c(0, 20, 25, 30, Inf), right = FALSE)
table(age_groups)
```

```
age_groups
  [0,20)  [20,25)  [25,30) [30,Inf)
    1246     6167      433      393
```

4. Cross-tabulate two categorical variables

- **Slides:** source slide 19.
- **What we are doing:** Compare the distribution of majors across the recorded gender categories. A two-way table counts each major–gender combination rather than summarizing either variable alone.
- **How to read the result:** Rows represent majors and columns represent gender categories. The final row and column contain totals. These are counts, so a larger group may contribute more observations without having a larger within-group percentage.

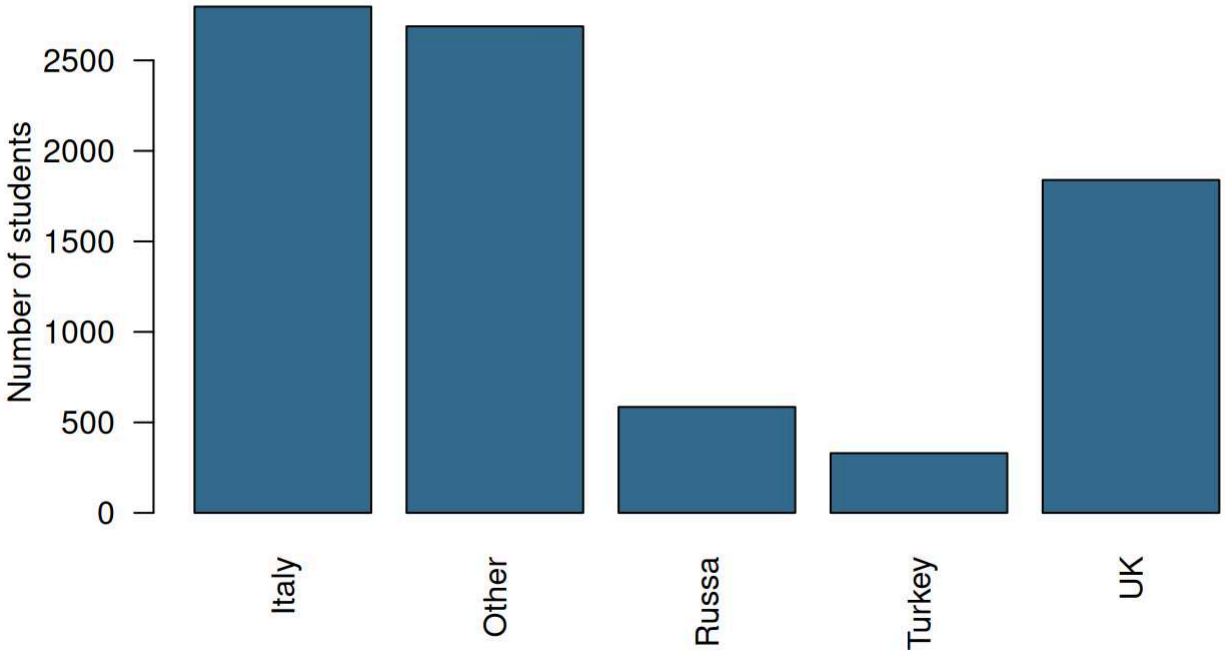
```
# xtabs() counts combinations: the formula ~ major + gender creates a two-way frequency table.
# addmargins() adds row and column totals without changing the underlying counts.
major_gender <- xtabs(~ major + gender, data = students)
addmargins(major_gender)
```

```
                          gender
major                      Female Male  Sum
  Biology                     959  638 1597
  Economics and Finance       461  863 1324
  Environmental Sciences      745  881 1626
  Mathematics and Statistics  276  949 1225
  Political Science           978  477 1455
  Social Sciences             691  321 1012
  Sum                        4110 4129 8239
```

5. Bar chart of category frequencies

- **Slides:** source slide 23.
- **What we are doing:** Display the same country frequencies as a bar chart. Bar height encodes the number of records in each country category; the horizontal positions identify categories rather than a numerical scale.
- **How to read the result:** Compare the bar heights with the frequency table. Rotated labels make the category names readable. Keep the distinction between a category count and a percentage of all records.

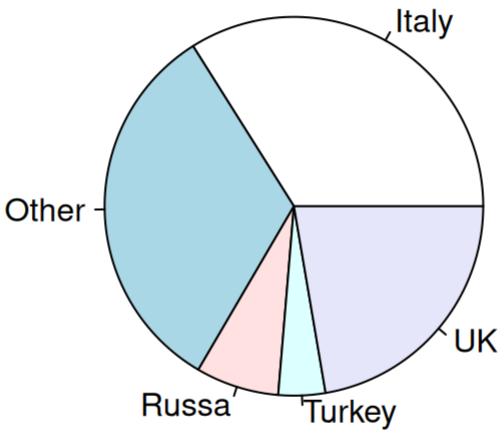
```
# barplot() draws a bar for each category count supplied by table().
# las=2 rotates country labels; ylab states that bar heights are numbers of students.
barplot(table(students$country), las = 2, col = "#356B8C", ylab = "Number of students")
```



6. Pie chart of category shares

- **Slides:** source slide 24.
- **What we are doing:** Show the country composition as parts of a whole. This uses the same country counts as the table and bar chart, allowing us to compare different displays of the same information.
- **How to read the result:** Each slice represents its share of all recorded country categories, including Other. Use the frequency table when precise comparisons are needed; similarly sized slices can be difficult to compare visually.

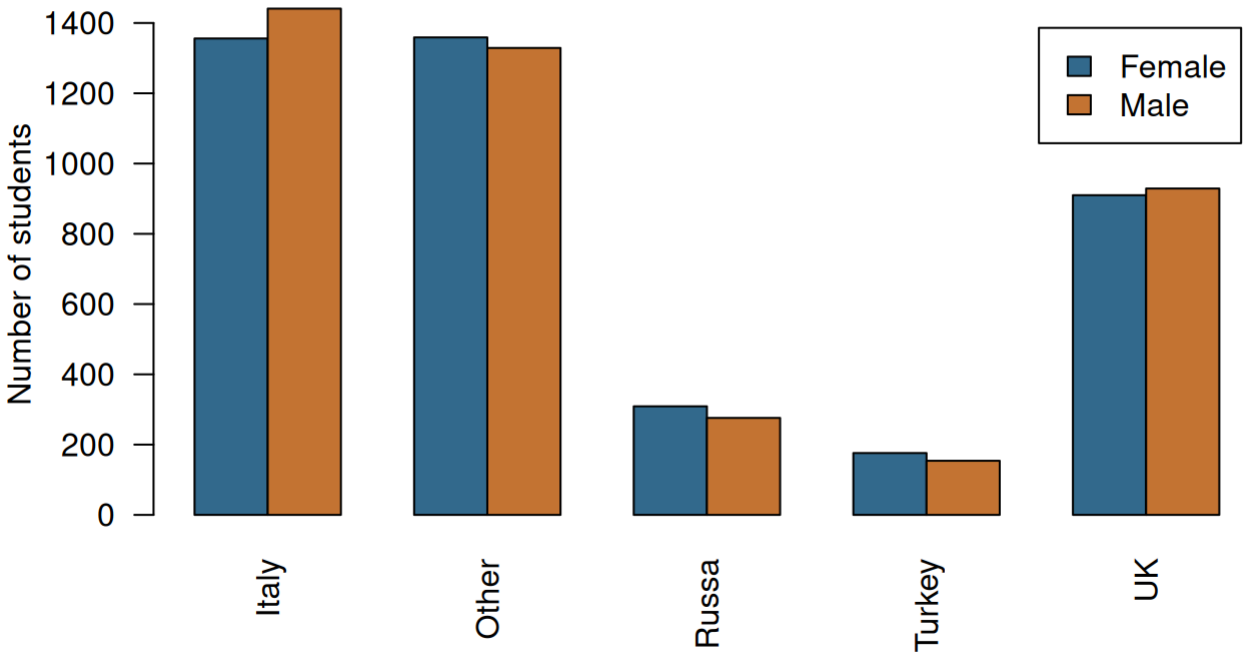
```
# pie() divides the circle according to the country counts computed by table().
# The denominator is all students with a recorded country category, including Other.
pie(table(students$country))
```



7. Grouped bars for two categorical variables

- **Slides:** source slide 26.
- **What we are doing:** Compare country counts across the recorded gender categories. Within each country, the bars are placed side by side so that the groups can be compared.
- **How to read the result:** Use the legend to identify gender categories and the vertical axis to read counts. Unequal overall group sizes affect the heights: these bars do not show within-gender or within-country percentages.

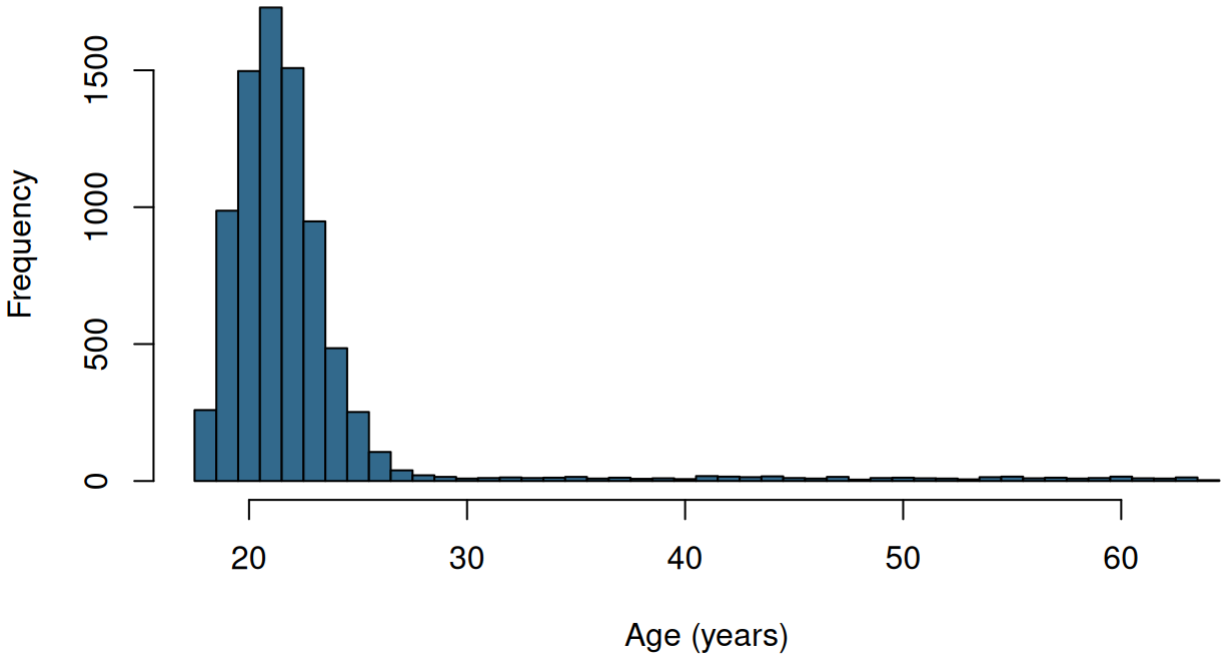
```
# table() cross-classifies gender and country; barplot(beside=TRUE) places gender bars side by side.
# legend.text=TRUE labels the groups; counts, rather than percentages, determine bar heights.
barplot(table(students$gender, students$country), beside = TRUE, legend.text = TRUE,
        col = c("#356B8C", "#C47735"), ylab = "Number of students", las = 2)
```



8. Histogram of ages

- **Slides:** source slide 28.
- **What we are doing:** Examine the distribution of age as a numerical variable. A histogram groups observations into adjacent numerical intervals and displays how many records fall into each bin.
- **How to read the result:** The boundaries lie halfway between recorded ages, giving one-year bins. Look for the most common ages, the overall spread and any long tail. The vertical axis shows frequencies, not probability density.

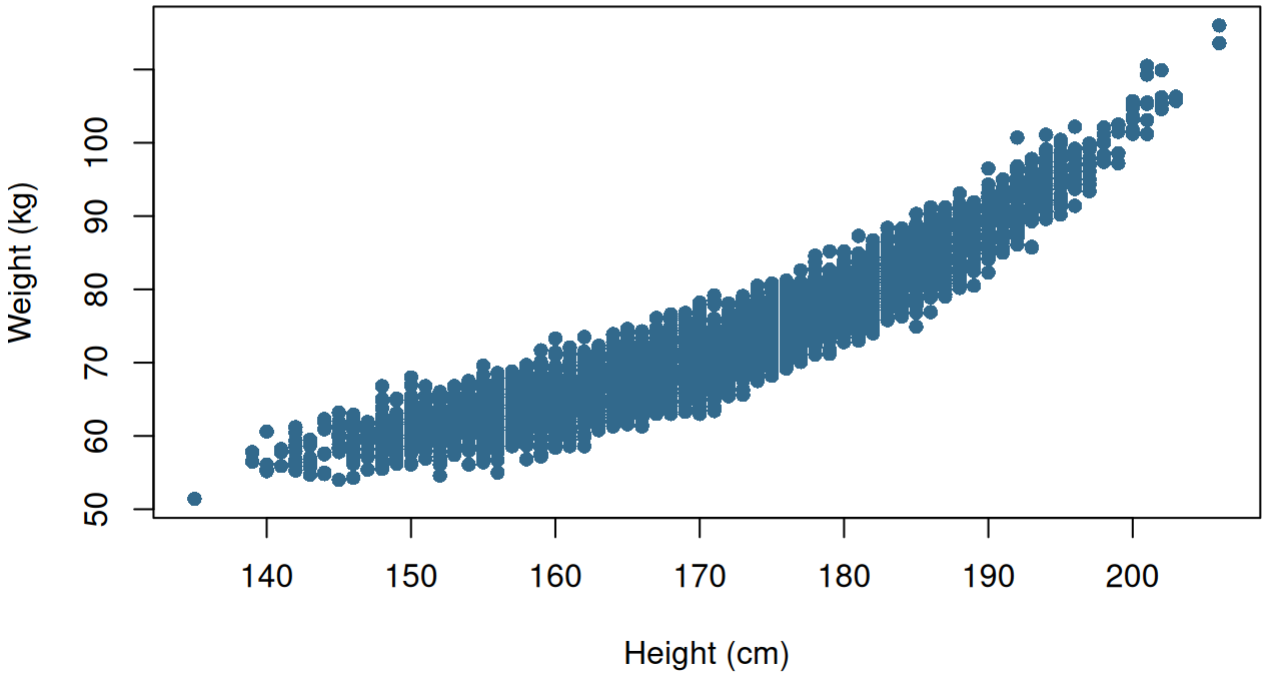
```
# hist() groups ages into bins; breaks specifies yearly boundaries at half-year positions.
# freq=TRUE displays counts rather than probability density; recorded ages are measured in years.
hist(students$age, breaks = seq(min(students$age) - 0.5, max(students$age) + 0.5, by = 1),
     freq = TRUE, col = "#356B8C", main = "", xlab = "Age (years)")
```



9. Scatterplot of height and weight

- **Slides:** source slide 33.
- **What we are doing:** Examine whether taller records tend to have greater recorded weight. Each point represents a paired height and weight from one row of the dataset.
- **How to read the result:** Read height on the horizontal axis and weight on the vertical axis. Look for direction, form, strength and unusual points. A visual association does not establish that changing height causes a change in weight.

```
# plot(x,y) makes a scatterplot, with height on the horizontal axis and weight on the vertical.
# pch=16 uses filled points; no fitted line or causal relationship is assumed.
plot(students$height, students$weight, pch = 16, col = "#356B8C",
     xlab = "Height (cm)", ylab = "Weight (kg)")
```



10. Mean, median and modes

- **Slides:** source slide 42.
- **What we are doing:** Summarize the centre of the age distribution in three ways. The mean uses every value, the median is the middle ordered value, and a mode is any recorded age with the largest frequency.
- **How to read the result:** The last output can contain more than one age if the highest frequency is tied. Compare the mean and median with the histogram; extreme ages can affect the mean more strongly than the median.

```
# mean() and median() summarize ages; table() counts each recorded age to identify every tied mode.
# names() returns the ages whose counts equal max(), the greatest frequency.
mean(students$age)
```

```
[1] 22.54157
```

```
median(students$age)
```

```
[1] 21
```

```
age_counts <- table(students$age)
names(age_counts)[age_counts == max(age_counts)]
```

```
[1] "21"
```

11. Range, mean absolute deviation, variance and standard deviation

- **Slides:** source slide 50.

- **What we are doing:** Describe how widely the recorded ages vary. The range uses the two endpoints; mean absolute deviation averages absolute distances from the mean; sample variance and standard deviation use squared distances with denominator n−1.
- **How to read the result:** Range, mean absolute deviation and standard deviation are in years; variance is in squared years. Here MAD means mean absolute deviation. R's `mad()` has a different definition, so the code explicitly calculates `mean(abs(age - mean(age)))`.

```
# diff(range()) gives maximum minus minimum; mean(abs()) calculates mean absolute deviation.
# var() and sd() use the sample denominator n-1; R's mad() instead computes median absolute deviation.
age <- students$age
c(range = diff(range(age)), mean_absolute_deviation = mean(abs(age - mean(age))),
  sample_variance = var(age), sample_standard_deviation = sd(age))
```

```
                   range   mean_absolute_deviation           sample_variance
               46.000000                  2.902923                 36.785568
sample_standard_deviation
                6.065111
```

12. Coefficient of variation

- **Slides:** source slide 55.
- **What we are doing:** Compare relative variability for the two exact datasets shown in the slides. The second dataset adds 1,000 to every value, keeping the sample standard deviation unchanged while increasing the mean.
- **How to read the result:** Each output is the sample standard deviation divided by the mean, expressed as a ratio. Multiply by 100 to express it as a percentage. These positive-mean examples illustrate the calculation; substantive CV comparisons require a meaningful ratio scale and an appropriate nonzero mean.

```
# c() creates the exact two datasets shown in the original table; sd() uses the sample denominator n-
        1.
# sd()/mean() expresses CV as a ratio, without multiplying by 100; these means are positive.
DS1 <- c(10, 20, 30, 40, 50)
DS2 <- c(1010, 1020, 1030, 1040, 1050)
c(DS1 = sd(DS1) / mean(DS1), DS2 = sd(DS2) / mean(DS2))
```

```
       DS1        DS2
0.52704628 0.01535086
```

13. Covariance and Pearson correlation

- **Slides:** source slide 63.
- **What we are doing:** Measure linear association for height–weight and height–age pairs. Covariance retains the variables' combined units; correlation standardizes the measure onto a scale from −1 to +1.
- **How to read the result:** Outputs appear in this order: height–weight covariance, height–weight correlation, height–age covariance and height–age correlation. `complete.obs` uses rows with both variables recorded for each pair. Association alone does not establish causation, and correlation can miss nonlinear patterns.

```
# cov() calculates sample covariance and cor() calculates Pearson's linear correlation by default.
# use="complete.obs" omits pairs with missing values; units remain cm, kg, and years.
cov(students$height, students$weight, use = "complete.obs")
```

```
[1] 89.89815
```

```
cor(students$height, students$weight, use = "complete.obs")
```

```
[1] 0.9398042
```

```r
cov(students$height, students$age, use = "complete.obs")
```

```
[1] -0.8861261
```

```r
cor(students$height, students$age, use = "complete.obs")
```

```
[1] -0.01318906
```

14. Adjusted sample skewness

- **Slides:** source slide 68.
- **What we are doing:** Quantify the asymmetry of the age distribution with the adjusted sample skewness used by Excel SKEW. The code standardizes each age using the sample standard deviation, cubes those standardized distances and applies the sample-size correction.
- **How to read the result:** A positive value indicates a longer right tail; a negative value indicates a longer left tail. Compare the sign with the histogram rather than treating this statistic as a complete description of shape. The formula requires more than two observations and nonzero variation.

```r
# length() supplies n; sum() adds cubed standardized deviations, using sample sd() with n-1.
# n/((n-1)*(n-2)) gives adjusted sample skewness, corresponding to Excel SKEW for n>2.
age <- students$age
n <- length(age)
n / ((n - 1) * (n - 2)) * sum(((age - mean(age)) / sd(age))^3)
```

```
[1] 4.427823
```

15. The 90th percentile

- **Slides:** source slide 72.
- **What we are doing:** Find the height value associated with the 90th percentile. Percentiles locate values within the ordered data; interpolation conventions can produce different numerical answers.
- **How to read the result:** The first output uses R type 7, corresponding to Excel PERCENTILE.INC. The second uses type 6, corresponding to PERCENTILE.EXC for this sample and probability. Both outputs are in centimetres. State which convention you use.

```r
# quantile() finds the requested percentile; probs=0.90 means the 90th percentile of height in cm.
# type=7 matches Excel PERCENTILE.INC; type=6 matches PERCENTILE.EXC for this sample and probability.
quantile(students$height, probs = 0.90, type = 7)
```

```
90%
186
```

```r
quantile(students$height, probs = 0.90, type = 6)
```

```
90%
186
```

16. First and second quartiles

- **Slides:** source slide 74.
- **What we are doing:** Locate the 25th and 50th percentiles of height. These are the first quartile and median, respectively, and divide the ordered observations into interpretable portions.
- **How to read the result:** Each output contains Q1 and Q2 in centimetres. Compare the inclusive and exclusive conventions. The chosen probabilities are valid for both conventions with this dataset; differences reflect interpolation rules rather than a change in the observations.

```
# quantile(probs=c(0.25,0.50)) returns the first and second quartiles of height in cm.
# type=7 uses Excel's inclusive convention; type=6 uses its exclusive convention for these
        probabilities.
quantile(students$height, probs = c(0.25, 0.50), type = 7)
```

```
25% 50%
163 171
```

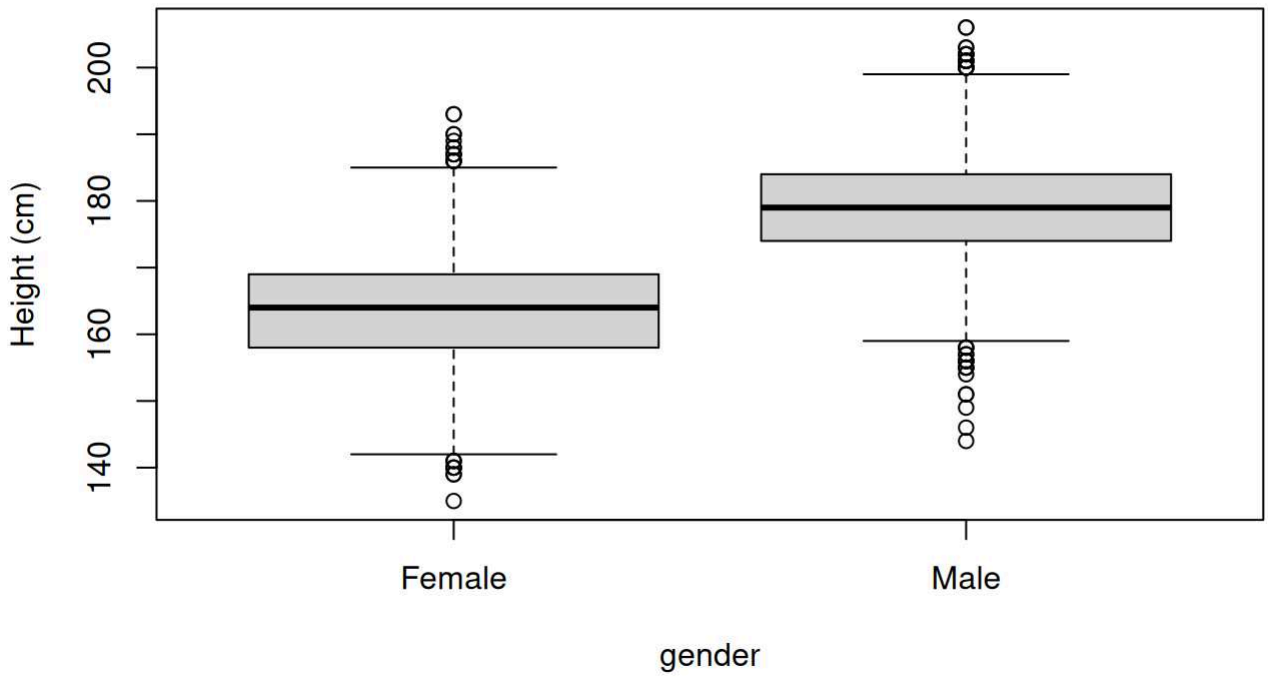
```
quantile(students$height, probs = c(0.25, 0.50), type = 6)
```

```
25% 50%
163 171
```

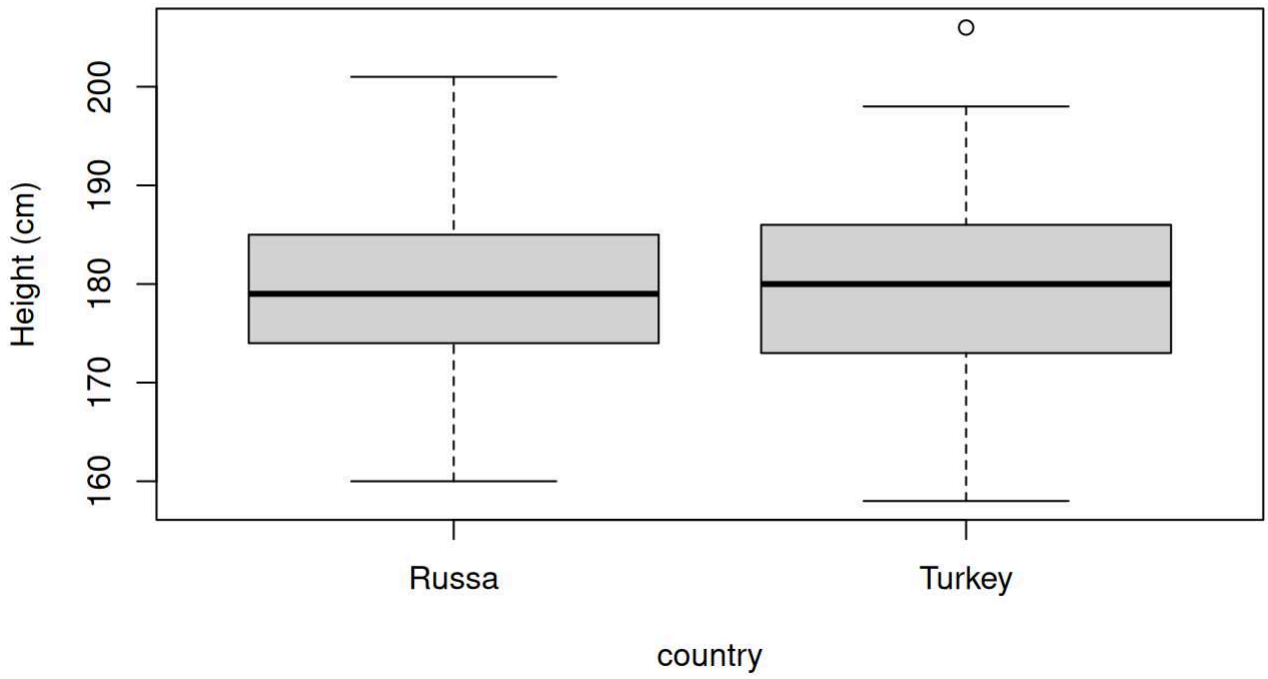
17. Boxplots and group comparisons

- **Slides:** source slide 76.
- **What we are doing:** Compare height distributions first by gender and then, among males, by the two selected country labels. Boxplots summarize the centre and spread and mark observations beyond the whiskers.
- **How to read the result:** Read the median line, box and whiskers together. R uses boxplot hinges, which can differ from type-7 quartiles; whiskers reach the most extreme observations within 1.5 IQR of the hinges. The original label `Russa` is retained for the Russian category. Group differences here are descriptive, with no causal or population inference implied.

```
# boxplot(height~gender) compares groups; whiskers extend to the last data within 1.5 IQR of the
        hinges.
# subset() selects Turkish/Russian males; R's default boxplot hinges can differ from type-7 quartiles.
boxplot(height ~ gender, data = students, ylab = "Height (cm)", col = "#D4D4D4")
```




```r
# The source spells Russia as "Russa"; match that original label without changing the data.
males <- subset(students, gender == "Male" & country %in% c("Turkey", "Russa"))
boxplot(height ~ country, data = males, ylab = "Height (cm)", col = "#D4D4D4")
```



18. Use IQR fences to flag potential outliers

- **Slides:** source slide 79.
- **What we are doing:** Calculate inclusive first and third age quartiles, their interquartile range and the lower and upper 1.5-IQR fences. Then count records outside those fences.
- **How to read the result:** The first output gives the fences in years; the second gives the number of flagged records. The strict comparisons flag ages below the lower fence or above the upper fence. Investigate flagged values in context; the code preserves every observation and does not automatically remove records.

```
# quantile(type=7) supplies inclusive quartiles; the 1.5-IQR fences flag values for investigation.
# sum() counts TRUE flags and does not remove observations; these age fences are measured in years.
q <- quantile(students$age, probs = c(0.25, 0.75), type = 7)
iqr <- q[2] - q[1]
lower <- q[1] - 1.5 * iqr
upper <- q[2] + 1.5 * iqr
c(lower_fence = unname(lower), upper_fence = unname(upper))
```

```
lower_fence upper_fence
       15.5        27.5
```

```
sum(students$age < lower | students$age > upper)
```

```
[1] 429
```

Review together

- Which tables summarize a single variable, and which compare two variables?
- Which charts suit categorical data, and which suit numerical data?
- How do the mean and median describe different aspects of the same distribution?
- Which dispersion measures retain the original units, and which do not?
- Why should percentile conventions be stated when comparing R and Excel results?
- Why does an outlier flag call for investigation rather than automatic deletion?
- Return to the Part 2 slides for the definitions and Excel instructions. For a separate business application, use the Superstore extended case.